

Documentación de Endpoints REST con Swagger (OpenAPI)

Proyecto: Sistema de Monopatines, Usuarios, Viajes, Facturación y Mantenimiento.

Esta documentación describe los endpoints REST del sistema utilizando el estándar OpenAPI (Swagger), incluyendo la configuración básica, seguridad con JWT y el listado completo de rutas expuestas por los microservicios.

1. Introducción a Swagger / OpenAPI

Swagger (OpenAPI) es un estándar para documentar APIs REST. En este proyecto se utiliza springdoc-openapi para generar automáticamente la documentación y exponerla en Swagger UI.

La documentación de la API permite visualizar en una interfaz web todos los endpoints, sus métodos HTTP, parámetros, códigos de respuesta y el esquema de seguridad basado en JWT (Bearer token).

2. Configuración de OpenAPI en el proyecto

2.1 Dependencia Maven (springdoc-openapi)

```
org.springdoc
springdoc-openapi-starter-webmvc-ui
2.6.0
```

2.2 Configuración de OpenAPI y esquema de seguridad JWT

```
@Configuration
@OpenAPIDefinition(
    info = @Info(
        title = "API Monopatines",
        version = "v1",
        description = "Microservicios de cuentas, usuarios, viajes, monopatines, paradas, m
antenimiento, facturación, tarifas y autenticación."
    ),
    security = {
        @SecurityRequirement(name = "bearerAuth")
    }
)
@SecurityScheme(
    name = "bearerAuth",
    description = "JWT Bearer token. Formato: 'Bearer {token}'",
    type = SecuritySchemeType.HTTP,
    bearerFormat = "JWT",
    scheme = "bearer",
    in = SecuritySchemeIn.HEADER
)
public class OpenApiConfig {
}
```

Con esta configuración, todas las operaciones de la API quedan protegidas por defecto mediante un esquema de seguridad tipo Bearer JWT, salvo los endpoints públicos que se marcan explícitamente como sin seguridad.

3. Seguridad y roles (resumen)

La seguridad se configura con Spring Security y JWT. Los roles principales utilizados son:

ADMIN: puede acceder a reportes, ajustes de tarifas, totales facturados, ranking de usuarios, etc.

USER: puede consultar información como monopatines cercanos, uso de cuenta, etc.

Los endpoints de autenticación /auth/** son públicos (no requieren token). En Swagger se marcan como públicos sobrescribiendo la seguridad en cada operación.

```
@Operation(
    summary = "Login de usuario",
    description = "Autentica un usuario y devuelve un JWT.",
    security = { @SecurityRequirement(name = "") } // público
)
@PostMapping("/login")
public Map login(@RequestBody LoginRequest dto) { ... }
```

4. Endpoints documentados (por recurso)

4.1 Autenticación (AuthController)

Tag: Autenticación. Endpoints públicos para registro y login de usuarios.

```
POST /auth/login
- Descripción: Login de usuario. Recibe credenciales y devuelve un JWT.
- Body: LoginRequest (username, password)
- Respuestas:
    200 OK: Map con campo 'token'
POST /auth/register
- Descripción: Registro de nuevo usuario.
- Body: UserDTO
- Respuestas:
    200 OK: Usuario creado
```

4.2 Cuentas (CuentaController)

```
GET /cuentas
- Descripción: Lista todas las cuentas.
- Rol: Cualquier usuario autenticado.
GET /cuentas/{id}
- Descripción: Obtiene una cuenta por su ID.
- Path variable: id (Long)
POST /cuentas/crear/{idAdmin}
- Descripción: Crea una nueva cuenta. La operación es realizada por un administrador.
- Path variable: idAdmin (Long)
- Body: CuentaDTO
PUT /cuentas/{id}
- Descripción: Actualiza los datos de una cuenta.
- Path variable: id (Long)
- Body: CuentaDTO
DELETE /cuentas/borrar/{id}
- Descripción: Elimina lógicamente o físicamente una cuenta (según implementación).
- Path variable: id (Long)
PUT /cuentas/{id}/anular
- Descripción: Anula una cuenta.
- Path variable: id (Long)
- Rol: ADMIN
PUT /cuentas/{id}/activar
- Descripción: Activa una cuenta previamente anulada.
- Path variable: id (Long)
POST /cuentas/{idCuenta}/asociar-usuario/{idUser}
- Descripción: Asocia un usuario a una cuenta.
- Path variables: idCuenta (Long), idUsuario (Long)
PUT /cuentas/{id}/debitar/{monto}
- Descripción: Debita un monto del saldo de la cuenta.
- Path variables: id (Long), monto (double)
```

4.3 Usuarios (UsuarioController)

```
GET /usuarios
- Descripción: Lista todos los usuarios.
GET /usuarios/{id}
- Descripción: Obtiene un usuario por ID.
POST /usuarios/crear/{idAdmin}
- Descripción: Crea un nuevo usuario (operación realizada por un admin).
```

- Path variable: idAdmin (Long)
- Body: UsuarioDTO
- PUT /usuarios/{id}
 - Descripción: Actualiza los datos de un usuario.
 - Path variable: id (Long)
 - Body: UsuarioDTO
- DELETE /usuarios/borrar/{id}/{idAdmin}
 - Descripción: Elimina un usuario.
 - Path variables: id (Long), idAdmin (Long)
- GET /usuarios/uso/{id}
 - Descripción: Obtiene el uso de un usuario en un periodo de fechas.
 - Path variable: id (Long)
 - Query params:
 - desde (LocalDate, formato YYYY-MM-DD)
 - hasta (LocalDate, formato YYYY-MM-DD)
 - incluirRelacionados (boolean, default=false)
 - Respuesta: UsoCuentaDTO
- GET /usuarios/por-tipo-cuenta
 - Descripción: Obtiene IDs de usuarios por tipo de cuenta.
 - Query param:
 - tipoCuenta (String)
 - Respuesta: List

4.4 Monopatines (MonopatinController)

- GET /monopatines
 - Descripción: Lista todos los monopatines.
- GET /monopatines/{id}
 - Descripción: Obtiene un monopatín por ID.
- POST /monopatines/crear
 - Descripción: Crea un nuevo monopatín.
 - Body: MonopatinDTO
- PUT /monopatines/{id}
 - Descripción: Actualiza un monopatín.
 - Body: MonopatinDTO
- DELETE /monopatines/borrar/{id}
 - Descripción: Elimina un monopatín por ID.
- PUT /monopatines/{id}/estado
 - Descripción: Cambia el estado de un monopatín.
 - Query param: estado (String)
- PUT /monopatines/{id}/finalizar
 - Descripción: Marca la finalización de un viaje en un monopatín.
 - Query params:
 - kmRecorridos (Double)
 - minutosTotales (Long)
- PUT /monopatines/{id}/ubicacion
 - Descripción: Actualiza la ubicación del monopatín.
 - Query params:
 - latitud (Double)
 - longitud (Double)
- GET /monopatines/cercanos
 - Descripción: Lista monopatines cercanos a una ubicación.
 - Query params:
 - lat (Double)
 - lon (Double)
 - radio (Double)
 - Rol: USER o ADMIN
- GET /monopatines/disponibilidad
 - Descripción: Obtiene información de disponibilidad general.
- GET /monopatines/reporte/km
 - Descripción: Reporte de km recorridos por monopatín, con o sin pausas.
 - Query param:
 - incluirPausas (boolean, default=false)
 - Rol: ADMIN

4.5 Paradas (ParadaController)

- GET /paradas
 - Descripción: Lista todas las paradas.
- GET /paradas/{id}
 - Descripción: Obtiene una parada por ID.
- POST /paradas/crear
 - Descripción: Crea una nueva parada.
 - Body: ParadaDTO

```
PUT /paradas/{id}
- Descripción: Actualiza una parada.
- Body: ParadaDTO
DELETE /paradas/borrar/{id}
- Descripción: Elimina una parada.
```

4.6 Mantenimientos (MantenimientoController)

```
GET /mantenimientos
- Descripción: Lista todos los mantenimientos.
GET /mantenimientos/{id}
- Descripción: Obtiene un mantenimiento por ID.
PUT /mantenimientos/{id}
- Descripción: Actualiza un mantenimiento.
- Body: MantenimientoDTO
POST /mantenimientos/iniciar
- Descripción: Inicia un mantenimiento real sobre un monopatín.
- Query params:
    monopatinId (Long)
    descripcion (String)
    responsable (String)
    costo (Double)
PUT /mantenimientos/{id}/finalizar
- Descripción: Finaliza un mantenimiento.
- Query params:
    observaciones (String)
    paradaId (Long, opcional)
GET /mantenimientos/activos
- Descripción: Lista solo los mantenimientos activos.
```

4.7 Viajes (ViajeController)

```
GET /api/v1/viajes
- Descripción: Lista todos los viajes (paginación desactivada en este endpoint).
GET /api/v1/viajes/{id}
- Descripción: Obtiene un viaje por ID (String, MongoDB).
POST /api/v1/viajes/iniciar
- Descripción: Inicia un nuevo viaje.
- Body: ViajeDTO
PUT /api/v1/viajes/{id}
- Descripción: Actualiza un viaje existente.
- Body: ViajeDTO
DELETE /api/v1/viajes/{id}
- Descripción: Elimina un viaje.
PUT /api/v1/viajes/{id}/cerrar
- Descripción: Cierra un viaje, calculando datos de facturación.
- Query param:
    kmRecorridos (Double)
- Respuesta: DatosDeFacturacionDTO
```

4.8 Reportes de Viajes

```
GET /api/v1/viajes/uso-cuenta/{idCuenta}
- Descripción: Uso de los viajes asociados a una cuenta en un período.
- Path variable: idCuenta (Long)
- Query params:
    desde (LocalDate, YYYY-MM-DD)
    hasta (LocalDate, YYYY-MM-DD)
- Respuesta: UsoDTO
GET /api/v1/viajes/uso-usuario/{idUsuario}
- Descripción: Uso de los viajes asociados a un usuario en un período.
- Path variable: idUsuario (Long)
- Query params:
    desde (LocalDate)
    hasta (LocalDate)
- Respuesta: UsoDTO
GET /api/v1/viajes/reporte/monopatines-mas-viajes
- Descripción: Monopatines con más de X viajes en un año.
- Query params:
    anio (int)
    minViajes (long)
```

- Rol: ADMIN

GET /api/v1/viajes/uso-usuarios-por-tipo

- Descripción: Ranking de usuarios más activos por tipo de usuario, en un período.
- Query params:
 - desde (LocalDate)
 - hasta (LocalDate)
 - tipoUsuario (String)
 - limite (int, default 0 = sin límite)
- Rol: ADMIN

4.9 Pausas de Viajes (PausaController)

GET /api/v1/viajes/pausas/{viajeId}

- Descripción: Lista todas las pausas de un viaje.
- Path variable: viajeId (String)

GET /api/v1/viajes/pausas/abierta/{viajeId}

- Descripción: Obtiene la pausa abierta de un viaje (si existe).

POST /api/v1/viajes/pausas/{viajeId}

- Descripción: Inicia una nueva pausa para el viaje indicado.

PUT /api/v1/viajes/pausas/cerrar/{viajeId}

- Descripción: Cierra la pausa abierta del viaje.

GET /api/v1/viajes/pausas/monopatines/{idMonopatin}/minutos

- Descripción: Obtiene minutos totales de pausa asociados a un monopatín.
- Path variable: idMonopatin (Long)
- Respuesta: MinutosPausaMonopatinDTO

4.10 Facturas (FacturaController)

POST /facturas/generar

- Descripción: Genera una factura a partir de los datos de un viaje.
- Body: DatosDeFacturacionDto
- Respuesta: Factura

GET /facturas/{id}

- Descripción: Obtiene una factura por ID.

GET /facturas/total

- Descripción: Calcula el total facturado entre meses indicados.
- Query params:
 - anio (int)
 - mesInicio (int)
 - mesFin (int)
- Rol: ADMIN

GET /facturas/todas

- Descripción: Obtiene todas las facturas del sistema.
- Rol: ADMIN

4.11 Tarifas (TarifaController)

GET /tarifas/vigente

- Descripción: Devuelve la tarifa vigente en una fecha determinada.
- Query param:
 - fecha (LocalDate, opcional; si falta, usa la fecha actual)

POST /tarifas/ajuste

- Descripción: Crea un nuevo ajuste de tarifa.
- Body: TarifaDto
- Rol: ADMIN

GET /tarifas/historial

- Descripción: Devuelve el historial de tarifas, de la más reciente a la más antigua.

DELETE /tarifas/{id}

- Descripción: Elimina una tarifa por ID.

4.12 IA (IaController)

POST /api/ia/prompt

- Descripción: Recibe un prompt en texto, la IA genera y ejecuta una consulta SQL sobre la base, devolviendo los resultados en JSON.
- Body: String (prompt)
- Respuesta: JSON con resultados o mensaje de error.

5. Conclusión

Este documento describe todos los endpoints REST del sistema utilizando la estructura y conceptos de Swagger/OpenAPI: paths, métodos HTTP, parámetros, cuerpos de petición, respuestas y esquema de seguridad con JWT. En el proyecto real, springdoc-openapi genera automáticamente la especificación OpenAPI y la expone en Swagger UI, permitiendo probar interactivamente cada endpoint con su correspondiente token JWT.